Demanda, Provisionamento e Desempenho

Grupo: Julio Cesar da Silva Alcântara - 122110504
Luana Lyz Araujo Rocha - 123110697
Sérgio de Sousa Duarte - 117110814

Aplicações desenvolvidas

São três servidores HTTP em Go (biblioteca padrão e cliente Prometheus) que diferem no recurso que esgotam primeiro: CPU, memória ou disco. Todos expõem `/livez` e `/readyz` (saúde), `/info` (CPUs que o processo enxerga) e `/metrics` (formato Prometheus). O endpoint principal recebe na query string quanto trabalho cada requisição faz, então o tempo de serviço se ajusta sem recompilar.

Aplicação	Porta	Endpoint	O que faz	Recurso
cpu	8081	`/cpu?iter=N`	SHA-256 aplicado N vezes em cadeia sobre 32 bytes	CPU
mem	8082	`/mem?mb=M&hold=H`	Aloca M MB, toca cada página de 4 KB, varre o buffer e o segura por H ms	RAM
io	8083	`/io?kb=K`	Grava K KB num arquivo novo, `fsync`, lê de volta e apaga	disco

Aplicação CPU-bound (cpu)

Descrição e características: GET `/cpu?iter=N` aplica o SHA-256 N vezes em cadeia, cada rodada sobre o resultado da anterior. Calibramos `iter=120000` para ~25 ms de CPU por requisição; no experimento a CPU por requisição medida ficou entre 25 e 32 ms. O runtime do Go distribui as requisições entre todas as CPUs que o processo enxerga (`GOMAXPROCS` igual ao número de vCPUs), então a aplicação usa todas as vCPUs provisionadas. No ensaio em Docker, com `iter=500000`, cada requisição consumiu ~29 ms de CPU e a vazão máxima dobrou a cada vez que as CPUs dobraram (37, 76 e 136 req/s com 1, 2 e 4 CPUs).

Monitoramento da aplicação e da infraestrutura: Quatro pontos de observação, ligados só durante cada nível de carga e alinhados pela janela de medição do gerador de carga:

1. **Cliente (k6):** registra cada requisição e exporta por nível a vazão, os percentis de latência (p50, p95, p99) e a taxa de erros. É o desempenho percebido por quem usa o serviço.
2. **Aplicação (`/metrics`):** lido a cada 2 s. Dali saem o tempo de CPU consumido pelo processo, a memória residente (RSS), o heap do Go, as requisições em andamento e um histograma da duração medido pelo servidor.
3. **Sistema convidado:** `vmstat` com uma amostra por segundo (fila de execução, CPU em modo usuário, sistema, ociosa e esperando IO, memória, swap), enviado ao host por ssh.
4. **Hipervisor (host):** a cada segundo, `VBoxManage metrics query` e a leitura do tempo de CPU e da memória do processo do VirtualBox em `/proc`. Mostra quanto a VM custa ao host, seja qual for o estado dentro dela.

Aplicação memory-bound (mem)

Descrição e características: GET `/mem?mb=M&hold=H` aloca um buffer de M MB, escreve um byte em cada página de 4 KB para o sistema reservar memória física, varre o buffer lendo uma posição a cada 64 bytes (uma linha de cache) e o mantém alocado por H ms antes de responder. Com c requisições simultâneas o conjunto de trabalho fica perto de c × M MB, então a RAM provisionada decide quando o sistema passa a usar swap ou mata o processo por falta de memória (OOM). No ensaio em Docker, com `mb=64&hold=200` e 2 CPUs, a memória em uso cresceu com a demanda até o limite: com 1 GB o processo passou a ser morto por falta de memória a partir de 32 usuários (66% de erros); com 2 GB, os erros começaram aos 32 usuários (5,6%) e o colapso veio aos 64 (83%). Com 1 GB, a memória chegou perto do teto já com 16 usuários, e o p95 subiu de 214 para 557 ms. Com 32 usuários, a vazão útil foi de 13,0 resp./s com 1 GB e de 42,9 com 2 GB: nesta aplicação, é a RAM que define a capacidade.

Monitoramento: Mesma instrumentação da aplicação de CPU: o k6 no cliente e o `/metrics` da aplicação. Como a carga dessa aplicação rodou em Docker, o coletor lê os contadores de cgroup do container no lugar do `vmstat` e das métricas do VirtualBox. Na VM, só verificamos o funcionamento (`lab.sh check` e `lab.sh demo`).

Aplicação IO-bound (io)

Descrição e características: GET `/io?kb=K` grava K KB num arquivo novo, chama `fsync`, lê o arquivo de volta e o apaga. O `fsync` só retorna quando o dispositivo confirma a escrita, então a requisição passa a maior parte do tempo esperando o disco, com a CPU ociosa. No ensaio em Docker, com `kb=256` e `fsync`, uma requisição levou 4 a 5 ms (p50), só 1 a 2,5 ms disso era CPU, e a CPU do container não passou de 7%. De 1 a 64 usuários, a vazão acompanhou a demanda (de 2,0 a 124,6 resp./s) e o p95 ficou perto de 7 ms. O disco virtual do Docker Desktop grava pelo cache do host, por isso a saturação não aparece nessa faixa de demanda. Esta aplicação foi testada numa única configuração de recursos.

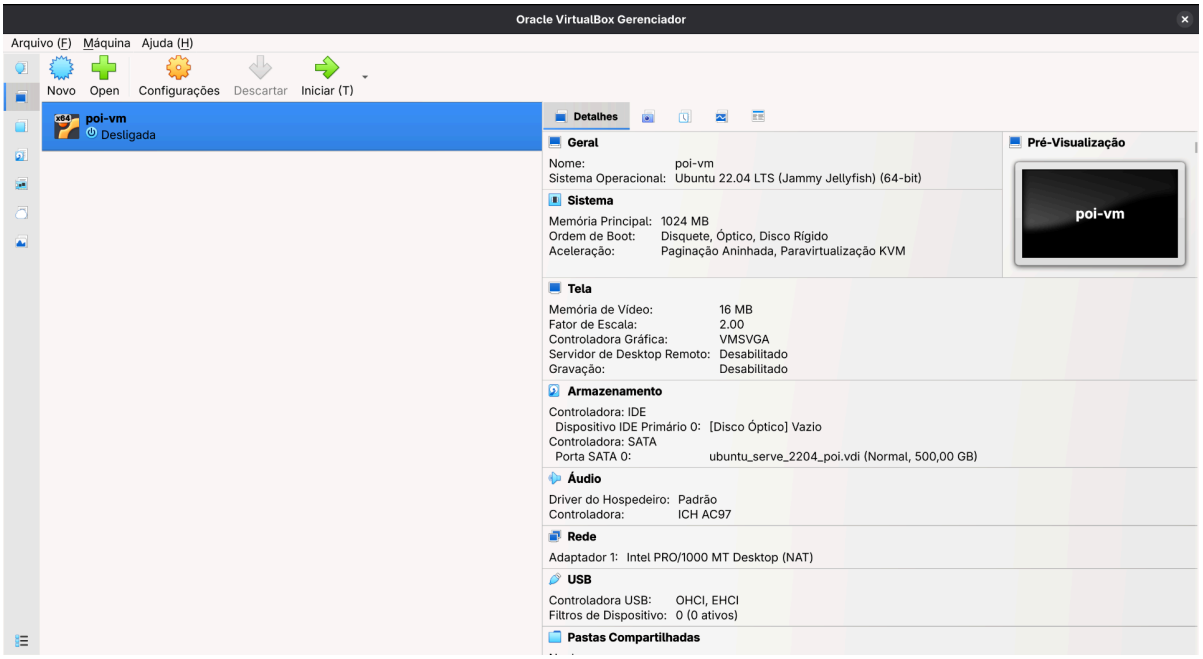
Monitoramento: Igual ao da aplicação de memória: k6, `/metrics` e cgroup do container; na VM, só a verificação de funcionamento.

Configuração da VM e Deploy da Aplicação

A VM `poi-vm` roda o Ubuntu 22.04 do VDI fornecido, com 1 vCPU e 1 GB de RAM na configuração base, e recebe as três aplicações como serviços systemd. Tudo foi feito num notebook (Fedora 43, Intel i7-1165G7 com 4 núcleos e 8 threads, 16 GB de RAM) com VirtualBox 7.2.20, e a carga é gerada no próprio hospedeiro, fora da VM.

Etapas de configuração da máquina virtual

1. Instalar o VirtualBox 7.2.20 (repositório da Oracle), os módulos do kernel (`kernel-devel` da versão em uso) e incluir o usuário no grupo `vboxusers`.
2. Copiar o VDI `ubuntu_serve_2204_poi.vdi` para uma pasta própria. No LCC o roteiro manda copiar de `/local/uasc/images` para `/tmp`.
3. Criar a VM `poi-vm` (Linux, Ubuntu 64-bit, 1 vCPU, 1024 MB, sem EFI) usando o disco existente, sem criar um novo.
4. Ligar a VM, entrar no console (osboxes) e conferir o servidor web e o ssh. A imagem já traz nginx 1.18.0 e OpenSSH 8.9p1; `curl localhost` na VM devolve a página padrão do nginx.
5. Configurar a rede NAT com cinco regras de redirecionamento de portas (tabela abaixo).
6. Configurar o acesso ssh por chave a partir do hospedeiro (`lab.sh ssh`, que usa `ssh-copy-id`).



Janela do VirtualBox com a `poi-vm` selecionada: Ubuntu 22.04 de 64 bits, 1024 MB de RAM, disco `ubuntu_serve_2204_poi.vdi` e rede NAT.

```
osboxes@osboxes:~$ dpkg -l nginx openssh-server
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name           Version          Architecture Description
+++-==============-================-============-=================================================
ii  nginx          1.18.0-6ubuntu14 amd64        small, powerful, scalable web/proxy server
ii  openssh-server 1:8.9p1-3        amd64        secure shell (SSH) server, for secure access fr
osboxes@osboxes:~$ curl -s localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

nginx 1.18.0 e openssh-server 8.9p1 instalados na VM, e `curl localhost` devolvendo a página padrão do nginx.

```
osboxes@osboxes:~$ sudo systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-10-14 12:15:02 UTC; 14min ago
       Docs: man:nginx(8)
    Process: 636 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exite
tatus=0/SUCCESS)
    Process: 688 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status
UCCESS)
   Main PID: 701 (nginx)
      Tasks: 2 (limit: 1034)
     Memory: 11.5M
        CPU: 64ms
     CGroup: /system.slice/nginx.service
             ├─701 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─706 "nginx: worker process" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" ""

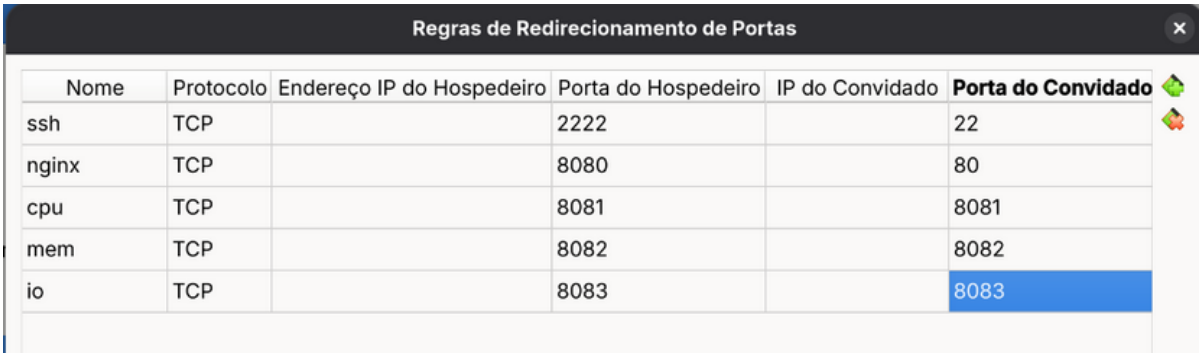
Oct 14 12:15:01 osboxes systemd[1]: Starting A high performance web server and a reverse prox…ve
Oct 14 12:15:02 osboxes systemd[1]: Started A high performance web server and a reverse proxy…er
Hint: Some lines were ellipsized, use -l to show in full.
```

`systemctl status nginx`: serviço ativo e habilitado no boot.

Nome	Porta no host	Porta na VM	Uso
ssh	2222	22	deploy e coleta do `vmstat`

nginx	8080	80	teste do roteiro
cpu	8081	8081	aplicação CPU-bound
mem	8082	8082	aplicação memory-bound
io	8083	8083	aplicação IO-bound

O nginx ficou na porta 8080 do host porque, no Linux, um usuário sem privilégios não abre portas abaixo de 1024.



Regras de redirecionamento de portas da interface NAT.

```
luana@fedora:~/ufcg/poi-atividade2$ bash lab/lab.sh ssh

== acesso ssh por chave
  esperando o sshd da VM na porta 2222...
  ssh-copy-id: digite a senha osboxes.org
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 4 key(s) remain to be installed -- if you are prompted now it is to install the new keys
osboxes@localhost's password:

Number of key(s) added: 4

Now try logging into the machine, with: "ssh -p 2222 -o 'StrictHostKeyChecking=accept-new' 'osboxes@localhost'"
and check to make sure that only the key(s) you wanted were added.

  ok      osboxes: 1 vCPU, 971 MB
luana@fedora:~/ufcg/poi-atividade2$
```

`lab.sh ssh`: *instala a chave pública na VM e confirma 1 vCPU e 971 MB vistos de dentro.*

Aplicação implantada e suas características

Escolhemos a aplicação CPU-bound para o teste de stress. A vazão dela depende só da CPU, então o efeito de mudar o número de vCPUs aparece direto nas medições, e um cenário que muda apenas a RAM serve de controle. As três aplicações foram implantadas e verificadas, mas só a de CPU foi submetida à varredura.

Etapas e requisitos do deploy da aplicação

O binário é compilado no hospedeiro para Linux x86-64 com `CGO_ENABLED=0`, o que gera um executável estático (Go 1.26.8). A VM não precisa de Go, de bibliotecas nem de pacotes além do sistema base. O `lab.sh deploy` copia o binário e a unit do systemd para a VM por ssh (porta 2222 do host) e executa `deploy/vm-setup.sh` com sudo. Esse script cria um usuário de sistema sem login (poi), instala o binário em `/opt/poi/bin` e habilita os serviços `poi-cpu`, `poi-mem` e `poi-io`. O systemd reinicia a aplicação se ela cair e a sobe a cada boot, então a VM pode ser reconfigurada e religada entre cenários sem intervenção manual.

Requisitos: acesso ssh por chave do host para a VM, as regras de redirecionamento de portas e permissão de sudo na VM para a instalação inicial.

```
luana@fedora:~/ufcg/poi-atividade2$ bash lab/lab.sh check

== acesso a partir do host
  ok      nginx (host 8080 -> VM 80)
          {"app":"cpu","go_version":"go1.26.8-X:nodwarf5","gomaxprocs":1,"hostname":"osboxes","num_cpu":1,"os_arch":"linux/amd64"}
  ok      8081/cpu?iter=200000 -> {"digest":"79fe15030e8e6d7c","elapsed_ms":56.019,"iter":200000}
          {"app":"mem","go_version":"go1.26.8-X:nodwarf5","gomaxprocs":1,"hostname":"osboxes","num_cpu":1,"os_arch":"linux/amd64"}
  ok      8082/mem?mb=64&hold=100 -> {"alloc_ms":57.089,"checksum":2088960,"elapsed_ms":163.162,"hold_ms":100,"mb":64,"scan_ms":5.712}
          {"app":"io","go_version":"go1.26.8-X:nodwarf5","gomaxprocs":1,"hostname":"osboxes","num_cpu":1,"os_arch":"linux/amd64"}
  ok      8083/io?kb=256 -> {"crc32":2545416815,"elapsed_ms":14.055,"fsync":true,"fsync_ms":13.095,"kb":256,"read_ms":0.197,"write_ms":0.504}
luana@fedora:~/ufcg/poi-atividade2$
```

`lab.sh check`: nginx (8080) e as três aplicações (8081, 8082, 8083) respondem a partir do hospedeiro. Cada `/info` mostra `gomaxprocs` e `num_cpu` iguais a 1.

```
luana@fedora:~/ufcg/poi-atividade2$ bash lab/lab.sh demo

$ VBoxManage list runningvms
"poi-vm" {a2543b11-adfb-4c0a-84bd-9f38f2dc0fa9}

$ ssh -p 2222 osboxes@localhost 'nproc; free -m | head -2; systemctl is-active poi-cpu poi-mem poi-io'
1
               total        used        free      shared  buff/cache   available
Mem:             971         264         376           1         331         560
active
active
active

$ curl -s localhost:8081/info; echo
{"app":"cpu","go_version":"go1.26.8-X:nodwarf5","gomaxprocs":1,"hostname":"osboxes","num_cpu":1,"os_arch":"linux/amd64"}


$ curl -s 'localhost:8081/cpu?iter=200000'; echo
{"digest":"79fe15030e8e6d7c","elapsed_ms":43.926,"iter":200000}


$ curl -s 'localhost:8082/mem?mb=64&hold=100'; echo
{"alloc_ms":52.839,"checksum":2088960,"elapsed_ms":158.625,"hold_ms":100,"mb":64,"scan_ms":5.605}


$ curl -s 'localhost:8083/io?kb=256'; echo
{"crc32":2545416815,"elapsed_ms":14.551,"fsync":true,"fsync_ms":13.759,"kb":256,"read_ms":0.178,"write_ms":0.362}


$ curl -s localhost:8081/metrics | grep -E '^(http_requests_total|process_resident_memory_bytes|process_cpu_seconds_total)'
http_requests_total{method="GET",path="GET /cpu",status="200"} 2
process_cpu_seconds_total 0.11
process_resident_memory_bytes 9.527296e+06
luana@fedora:~/ufcg/poi-atividade2$
```

`lab.sh demo`: VM ligada, os três serviços active, `/info`, uma chamada em cada aplicação e o `/metrics` da aplicação de CPU.

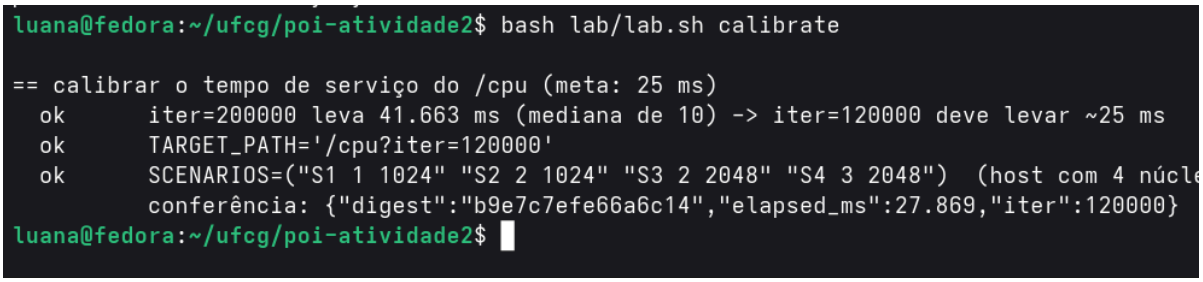
Teste de Stress da Aplicação selecionada

Comparamos quatro configurações de recursos da VM com carga gerada pelo k6 fora dela, em oito níveis de demanda, de 1 a 128 usuários simultâneos.

Cenários de configuração de recursos

Cenário	vCPUs	RAM	Objetivo
S1	1	1 GB	linha de base (configuração do roteiro)
S2	2	1 GB	efeito de dobrar a CPU
S3	2	2 GB	efeito de dobrar só a RAM, numa aplicação que não depende dela (controle)
S4	3	2 GB	escalabilidade com mais vCPUs, no limite do host

O host tem 4 núcleos físicos (8 threads), e o k6 e os processos do VirtualBox também precisam de CPU. Por isso o S4 usa 3 vCPUs, e não 4. O `lab.sh calibrate` ajustou o tempo de serviço: `iter=200000` levou 41,7 ms (mediana de 10 chamadas), então escolhemos `iter=120000` para ~25 ms; a conferência deu 27,9 ms.



`lab.sh calibrate`: tempo de serviço de `/cpu` e escolha de `iter=120000`.

Como os testes de stress foram realizados

A carga vem de fora da VM: o k6 roda no hospedeiro e acessa `http://localhost:8081/cpu?iter=120000`, que o NAT redireciona para a VM. Como o k6 divide os núcleos do notebook com a VM, esse efeito deve pesar mais nos cenários com mais vCPUs.

Cada usuário virtual envia uma requisição, espera a resposta, fica ocioso por um tempo aleatório com distribuição exponencial de média 0,5 s (o *think time*) e envia a próxima. É um modelo de usuários interativos em laço fechado, e a demanda é o número de usuários simultâneos: 1, 2, 4, 8, 16, 32, 64 e 128.

Cada nível dura 70 s: 10 s de aquecimento, descartados, e 60 s de medição. O `experiments/run.sh` automatiza a sequência. Para cada cenário ele desliga a VM por ACPI, aplica `VBoxManage modifyvm --cpus --memory`, liga a VM em modo headless, espera o `/readyz` responder, confere de dentro da VM se `nproc` e `free -m` batem com o cenário, espera 15 s para o sistema estabilizar e roda os oito níveis com os coletores descritos em "Aplicações desenvolvidas". A primeira rodada do S2 saiu perturbada: com 16 usuários o p50 foi de 67 ms, contra 30 ms na repetição, e a vazão não passou de 53 req/s. Repetimos o cenário inteiro e usamos os dados da repetição. O log da rodada descartada está no repositório, e não identificamos a causa.

Resultados comparativos entre os cenários

A vazão máxima sobe de 40 req/s com 1 vCPU para 74 com 2 e 96 com 3, e dobrar só a RAM (S2 para S3) não mudou nada: 73,6 contra 72,5 req/s. Nenhum nível teve erro (0% em todos os 32).

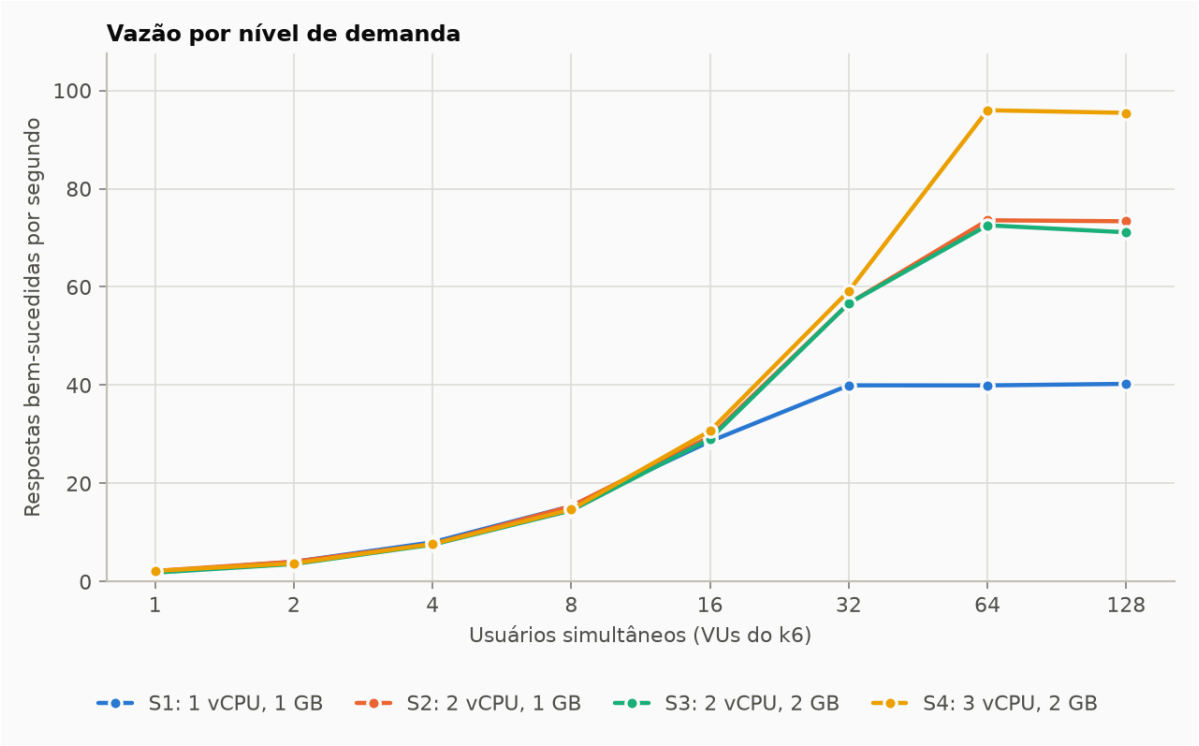
Cenário	vCPUs	RAM	CPU por requisição (ms)	Vazão máx. prevista (req/s)	Vazão máx. medida (req/s)	Medida / prevista	N* previsto (usuários)	p95 passa de 2× o valor sem carga em (usuários)
S1	1	1 GB	26,8	37,3	40,3	108%	19,7	8
S2	2	1 GB	28,6	69,9	73,6	105%	36,9	32
S3	2	2 GB	28,9	69,3	72,5	105%	36,6	32
S4	3	2 GB	30,3	99,0	96,0	97%	52,5	64

A vazão máxima prevista é vCPUs / CPU por requisição (lei da utilização), com a CPU por requisição medida em cada cenário. N* = vazão máxima × (CPU por requisição + 0,5 s de think time) é o número de usuários em que a CPU satura. A última coluna é o primeiro nível em que o p95 passa do dobro do p95 medido com 1 usuário. A tabela com todos os níveis está no repositório, em `docs/figuras/vbox-cpu/summary.md`.

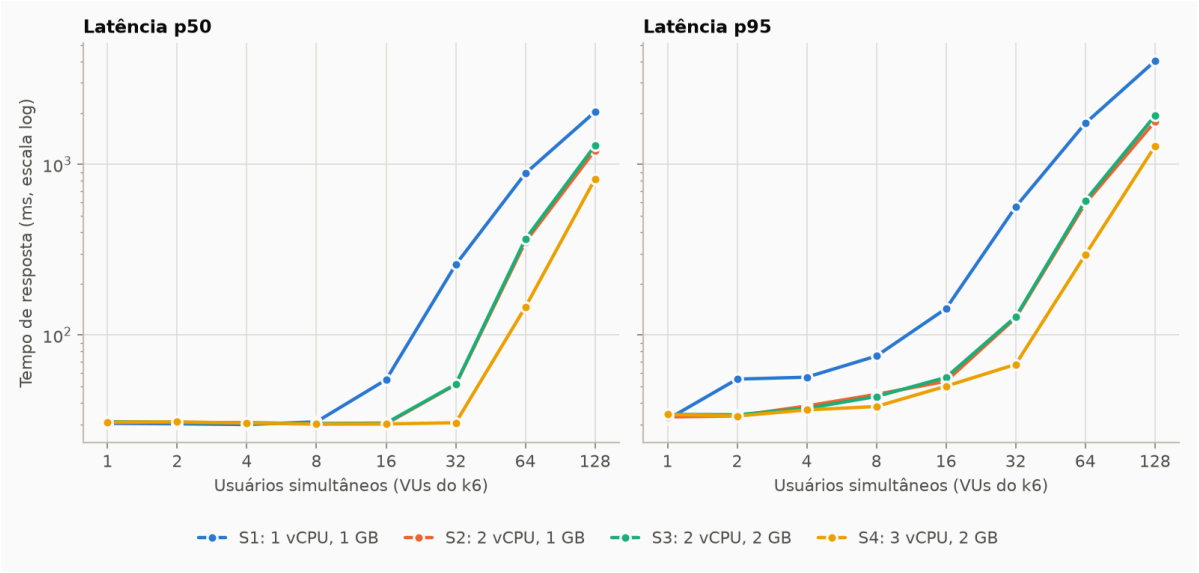
Consumo de recursos da infraestrutura (VM no VirtualBox). A tabela compara a CPU vista de dentro da VM (`vmstat`, em % das vCPUs) com a CPU que o processo do VirtualBox consumiu no hospedeiro (em % das vCPUs provisionadas), em quatro níveis de carga. A memória usada dentro da VM ficou entre 207 e 228 MB em todos os níveis e cenários, ou seja, a RAM provisionada (1 ou 2 GB) nunca foi pressionada.

Cenário	Medida	1 usuário	16 usuários	64 usuários	128 usuários
S1 (1 vCPU)	CPU da VM no hospedeiro (%)	11,1	77,6	101,8	102,3
S1 (1 vCPU)	CPU dentro da VM (%)	2,3	55,8	100,0	100,0
S2 (2 vCPUs)	CPU da VM no hospedeiro (%)	5,3	44,7	96,3	96,3
S2 (2 vCPUs)	CPU dentro da VM (%)	2,6	37,1	94,9	94,6
S3 (2 vCPUs)	CPU da VM no hospedeiro (%)	4,8	44,2	95,9	95,5
S3 (2 vCPUs)	CPU dentro da VM (%)	2,4	37,3	94,3	93,6
S4 (3 vCPUs)	CPU da VM no hospedeiro (%)	3,9	33,4	85,3	86,2
S4 (3 vCPUs)	CPU dentro da VM (%)	2,3	29,9	83,8	83,8

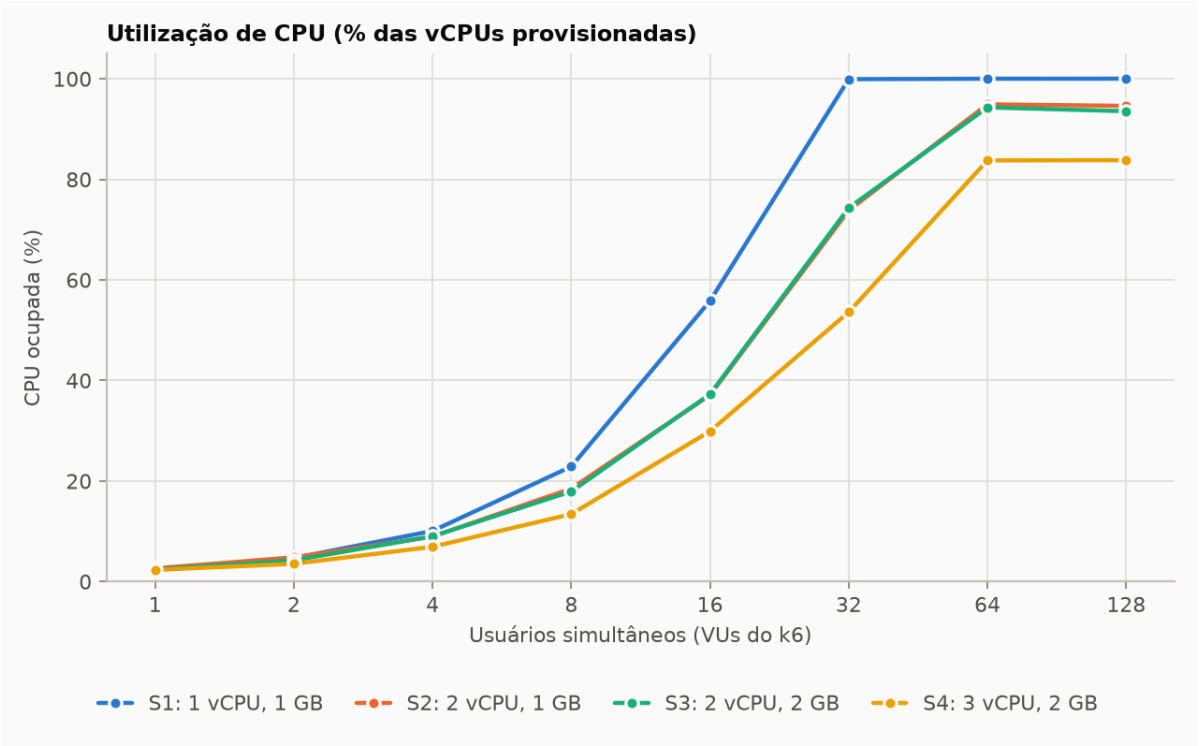
Com a VM quase ociosa (1 usuário), o processo do VirtualBox consome cerca de 0,1 núcleo do hospedeiro em todos os cenários (0,11, 0,11, 0,10 e 0,12 núcleo em S1 a S4), entre 0,05 e 0,09 núcleo a mais do que a VM relata. Essa diferença é o custo do próprio VirtualBox (emulação de dispositivos, temporizadores e threads auxiliares), que a VM não enxerga. Na saturação as duas medidas convergem (102% em S1, ~96% em S2 e S3). Em S4 o hospedeiro registra só 86% das três vCPUs mesmo com a demanda no máximo, o que concorda com a vazão estagnada em ~96 req/s: o limite ali parece estar no hospedeiro, que divide 4 núcleos físicos (8 threads) entre as três vCPUs, o k6 e o resto do sistema, e não no número de vCPUs configurado.



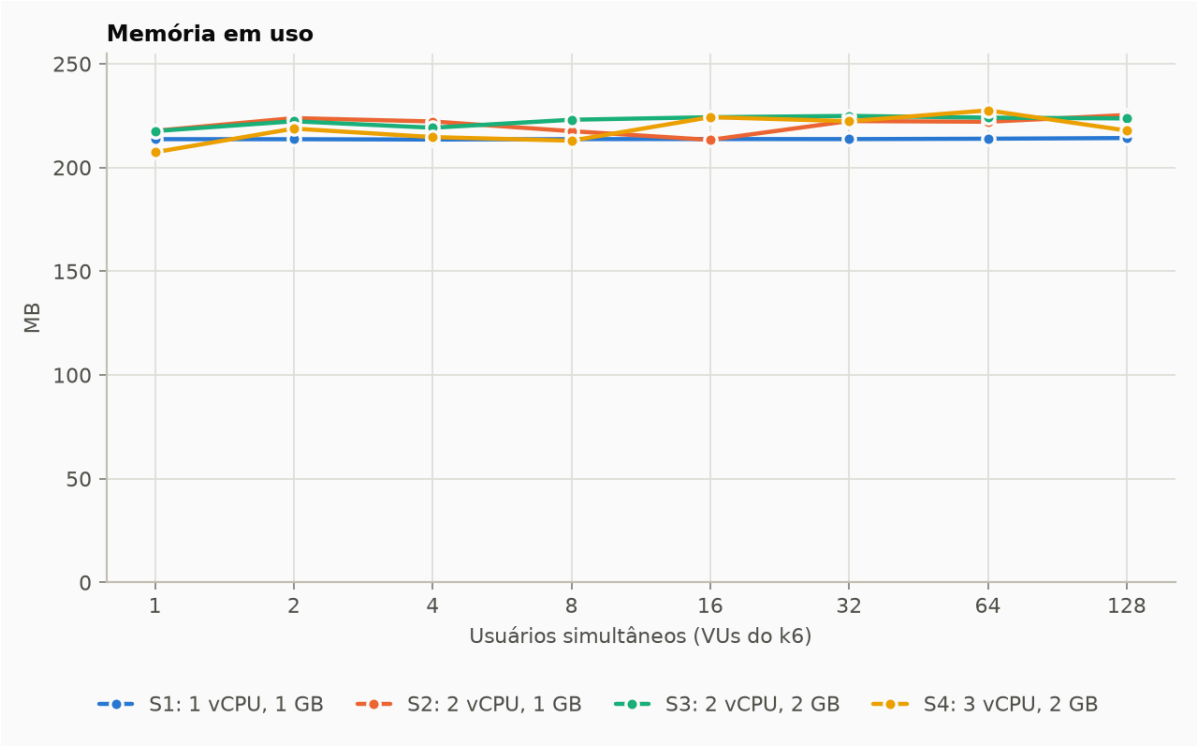
Vazão: o S1 estaciona em ~40 req/s a partir de 32 usuários; o S2 e o S3, em ~73 a partir de 64; o S4, em ~96 a partir de 64.



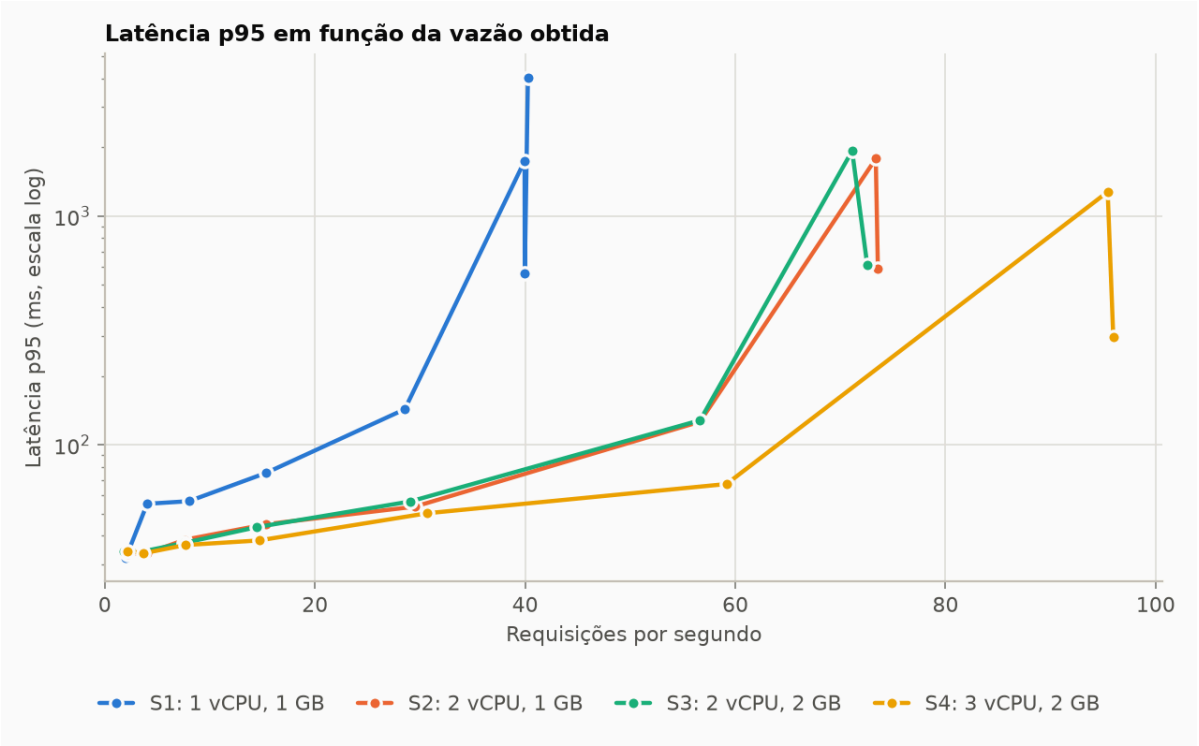
Latência (escala log): o p50 fica em ~30 ms enquanto a CPU não satura e depois cresce com a fila. No S1 com 128 usuários, o p50 é 2,0 s e o p95, 4,0 s.



CPU da VM, em % das vCPUs: o S1 chega a 100% com 32 usuários; o S2 e o S3 ficam em ~95% com 64 e 128; o S4, em 84%.



Memória usada na VM: entre 207 e 228 MB em todos os cenários e níveis, sem relação com a demanda. O processo da aplicação usa de 12 a 17 MB.



Cada ponto é um nível de demanda. Depois que a vazão satura, mais usuários só aumentam a latência, e os pontos sobem quase na vertical.

Interpretação e conclusões

Nesta aplicação a CPU é o único recurso que importa: cada vCPU a mais aumenta a vazão máxima quase na mesma proporção.

Onde cada cenário satura: A vazão para de crescer quando a CPU da VM chega perto do máximo, e dali em diante cada usuário a mais só alonga a fila. O S1 satura entre 16 e 32 usuários (28,6 e 40,0 req/s; CPU em 100% a partir de 32). O S2 e o S3 saturam entre 32 e 64 (56,6 e 73,6 req/s no S2, 56,6 e 72,5 no S3), e o S4 também (59,2 e 96,0). Isso bate com o N* previsto de 20, 37, 37 e 53 usuários. Nenhuma requisição falhou, porque a fila só espera.

O p95 piora antes da saturação: O p95 passa do dobro do valor sem carga em 8 usuários no S1, quando a CPU está em 23%, e só em 32, 32 e 64 usuários nos outros três. Hipótese, não testada: chegadas aleatórias em rajada, em que uma requisição espera a outra mesmo com a CPU folgada.

Quanto a vazão escala com as vCPUs: A vazão máxima medida foi 40,3, 73,6 e 96,0 req/s com 1, 2 e 3 vCPUs: 1,8× e 2,4× a de 1 vCPU, contra 2× e 3× do ideal. A previsão da lei da utilização acerta (medida entre 97% e 108% da prevista) porque usa a CPU por requisição de cada cenário, que sobe de 26,8 ms no S1 para 30,3 ms no S4 (+13%). Ou seja, cada requisição gasta mais CPU quando há mais vCPUs. Não medimos a causa. Hipóteses: SMT, disputa de núcleos com o k6 e o VirtualBox e variação da frequência do processador.

Por que o S3 não difere do S2: Os dois deram 72,5 e 73,6 req/s, CPU de 94,3% e 94,9% e p95 de 613 e 591 ms com 64 usuários. A aplicação usa 12 a 17 MB e a VM usa ~220 MB com qualquer carga, então sobram mais de 700 MB mesmo com 1 GB. Aumentar um recurso que não é o gargalo não melhora o desempenho. A diferença de 1,5% não dá para distinguir de variação normal, porque só há uma rodada válida por cenário.

Para dimensionar: Com p95 abaixo de 100 ms, o S1 atende até 8 usuários (15,4 req/s), o S2 e o S3 até 16 (29,5 e 29,1 req/s) e o S4 até 32 (59,2 req/s). Dobrar de 1 para 2 vCPUs dobrou essa capacidade. Para esta aplicação, a decisão de provisionamento é por CPU, e a RAM pode ficar no mínimo.

Limites desta análise: Foi um único host (notebook), com o k6 no mesmo computador da VM, uma rodada válida por cenário e só a aplicação de CPU. O modelo de capacidade usado só vale quando a CPU é o gargalo, então não se aplica às aplicações de memória e I/O, que só foram analisadas em Docker (a de I/O, numa única configuração de recursos). Os números valem para este hardware e este hipervisor.